

# Portable Spec Kit — User Guide

**Author:** Dr. Aqib Mumtaz · **License:** MIT · **GitHub:** [aqibmumtaz/portable-spec-kit](https://github.com/aqibmumtaz/portable-spec-kit)

A lightweight, zero-install, personalized framework for AI-assisted engineering. Drop one file into any project — your AI agent personalizes to you, maintains living specifications, and preserves context across sessions. Specs always exist. Always current. Never block.

## Quick start

Install into a new or existing project from a local kit checkout:

```
bash <kit-path>/install.sh --yes --from <kit-path>
```

Or one-shot network install:

```
curl -fsSL https://raw.githubusercontent.com/aqibmumtaz/portable-spec-kit/main
```

After install, every supported AI agent (Claude Code, Cursor, Copilot, Windsurf, Cline) reads the kit's rules immediately via the appropriate entry-point file.

## 5 reliability layers

| Layer                  | Type                  | What it stops                                               |
|------------------------|-----------------------|-------------------------------------------------------------|
| 2A — psk-sync-check.sh | Content (bash critic) | Structural drift across version anchors, counts, doc tables |

|                                     |                                             |                                                            |
|-------------------------------------|---------------------------------------------|------------------------------------------------------------|
| 2B — psk-critic-spawn.sh            | Content (semantic critic)                   | Stale content the bash checks can't see                    |
| 3 — Hooks (PostToolUse + PreCommit) | Content (outside agent control)             | Bypass of layers 2A/2B                                     |
| 4 — §Workflow Fidelity              | Process (behavioral, structurally enforced) | Agent overriding the kit's defined workflow under pressure |
| 5 — §Plan Execution Protocol        | Plan-shape (schema-gated driver)            | Agent improvising plan execution outside the schema        |

# Spec-Persistent Development (SPD) pipeline

6 pipeline stages + 3 support files: REQS → SPECS → PLANS → DESIGN → TASKS → RELEASES, with RESEARCH/AGENT/AGENT\_CONTEXT as cross-cutting support. Full R→F→T traceability: requirement → feature → test.

## Key features (v0.6.64)

- 36 reliability scripts (required) + 6 optional helpers
- 26 skill files (JIT-loaded)
- 30 flow documents
- 26 file-class templates (all pass 7-criterion Quality Bar)
- 2865 automated tests (2720 framework + 145 benchmarking)
- 12 mechanical reflex gates + 26 reflex audit dimensions

## Workflows (executable)

| Command | What it does |
|---------|--------------|
|---------|--------------|

|                                                                       |                                                               |
|-----------------------------------------------------------------------|---------------------------------------------------------------|
| <code>bash agent/scripts/psk-orchestrate.sh "your requirement"</code> | Full 10-phase pipeline (new project)                          |
| <code>bash agent/scripts/psk-orchestrate.sh --update</code>           | U0-U10 update cascade including U6.5 UI completeness backfill |
| <code>bash agent/scripts/psk-release.sh prepare</code>                | 10-step release ceremony with dual-gate validation            |
| <code>bash agent/scripts/psk-feature-complete.sh</code>               | Single-feature lifecycle with code review + sync              |
| <code>bash agent/scripts/psk-run-plan.sh start &lt;slug&gt;</code>    | Schema-aware plan executor (sub-agent per phase)              |
| <code>bash reflex/run.sh</code>                                       | Autoloop reflex audit until GRANTED                           |

## Documentation

- [Technical Overview](#) — architecture deep-dive + version changelog
- [Full Guide](#) — comprehensive operator manual
- [README](#) — installation + quick reference
- [CHANGELOG](#) — version history

## License

MIT License. See [LICENSE](#).